CSIE 5374 Assignment 1 (Due at 3/27 12:00 PM)

In assignment 1, you have to write a simple Linux kernel module and provide the following features: (1) hide/unhide module, (2) masquerade process name. The features are generally employed by a kernel rootkit, which is essentially a malware with kernel privileges. Additionally, you are asked to implement syscall filtering functionality in the module. This feature blocks sensitive system calls that are not used by the program to improve software safety.

In this assignment, you are asked to implement the module as a loadable kernel module (LKM). LKM runs in kernel mode and granted with access to all kernel internal structures/functions. It can be used to extend the functionality of the running kernel, and thus it is also often used to implement device drivers to support new hardware.

We provide an LKM template as a starting point for you. You should modify the module source to meet assignment requirements. You should also write a user space program to test the functionality of your rootkit module. Both the rookit and the test program must run on an AArch64 machine. We use QEMU to emulate this, as you did in `env-setup`.

Note that you are NOT allowed to modify the kernel source in this assignment. Your kernel module should work independently to the Linux kernel.

Development Tips

In the following sections, we assume you already know how to compile the Linux kernel and create a filesystem image and run it with `qemu-system-aarch64`. Therefore, we will skip those steps already mentioned in `env-setup`.

Compile LKM

To compile the kernel module, we assume you have downloaded or installed the following prerequisites.

- Prerequisite
  - Linux v6.1 source code
    - `git clone --depth 1 --branch v6.1 https://github.com/torvalds/linux.git`
  - Cross compiler
    - gcc-aarch64-linux-gnu (4:11.2.0-1ubuntu1)
    - `sudo apt update && sudo apt install gcc-aarch64-linux-gnu`
  - LKM template
    - rootkit

We have provided a Makefile in the LKM template directory, so you can switch to the LKM template directory and use the `make` command with the following arguments to build the kernel module.

```
cd /PATH/TO/rootkit
make KDIR=/PATH/TO/linux-6.1-source CROSS=aarch64-linux-gnu
```

(after `KDIR`, you should specify the path for your kernel source. e.g., `KDIR=~/src/linux-6.1/`)
After compilation, a kernel module binary with `.ko` extension is generated

NOTE 1: You have to make sure the kernel you use in your VM is compiled from the source you specified above.
NOTE 2: Before you test your module, you have to recompile your Linux to enable **kprobe**. See the **HINT 2** in the section **Filter syscall** for more details.

Install LKM

> PLEASE DO THIS STEP IN QEMU VM, DO NOT INSTALL IT ON YOUR HOST

After kernel module binary (i.e. `rootkit.ko`) is generated, you can install it to a running kernel using the `insmod` command.

```
sudo insmod rootkit.ko
```

You can check installed module using the `lsmod` command.

```
lsmod
```

In our template, we only implement `ioctl` file operation. However, you can also define other file operations as you want.

Uninstall LKM

When you want to update a kernel module, you need to remove the old one.
You can use `rmmod` to remove the installed module.

```
$ lsmod
Module                  Size  Used by
...
rootkit                XXXXX  0
...
```

```
rmmod rootkit
```

Requirements

In this assignment, you should modify the ***ioctl*** system call handler in **rootkit.c** to add new ***ioctl*** numbers for the four different functionality like the following:

```
    switch (ioctl) {
    case IOCTL_MOD_HIDE:
        // do something
        break;
    case IOCTL_MOD_MASQ:
        // do something
        break;
    case IOCTL_ADD_FILTER:
        // do something
        break;
    case IOCTL_REMOVE_FILTER:
        // do something
        break;
    default:
        ret = -EINVAL;
    }
```

Hide/Unhide module (10%)

To prevent others from discovering this rootkit through `lsmod`, you must implement a function to remove/add this module from the module list.

The rootkit module should be visible by default. Calling ioctl for the hide functionality hides the module (you will not see it from `lsmod`); if the module is hidden, making the same ioctl call unhides the module.

HINT 1: you can access module list via `THIS_MODULE->list`

Masquerade process name (30%)

Sometimes it's useful to be able to masquerade the name of a process as another process.

In this requirement, a parameter needs to be passed to the module so that the module knows which process should be renamed and what the new name is. This is what `struct masq_proc` is for.

However, you are asked to handle multiple `struct masq_proc` in a single `ioctl` operation. Hence, you must pass `struct masq_proc_req *` to the rootkit module.

The `list` field in `struct masq_proc_req` points to an array of `struct masq_proc`, and the `len` field indicates how many entries are in the `list`.

```
#define MASQ_LEN 20
struct masq_proc {
    char new_name[MASQ_LEN];
    char orig_name[MASQ_LEN];
};
struct masq_proc_req {
    size_t len;
    struct masq_proc *list;
};
```

In the module, you are required to handle an arbitrary `len`. You should not reserve a fixed-size array to store the contents. You should allocate memory according to the actual data size. You can use the function **kmalloc** to allocate memory, and **kfree** to free the memory allocated by kmalloc.

You should only masquerade the process name if the actual length of the **new_name** string is shorter than the **orig_name**. You are asked to iterate the list of **struct masq_proc**, and try to masquerade the process name using the data from each of the entries if possible.

HINT 1: If the function succeeds, you should observe the results from **ps ao pid, comm**.

Filter syscall (40%)

Implementation (30%)

In this section, you are required to implement syscall filtering to block specific system calls invoked by the process or thread. Your implementation must provide a mechanism for specifying which system calls should be blocked for the process. This is achieved using a filtering structure that includes both the system call number and the process name.

This functionality is managed using the following structure:

```
TASK_FILTER_LEN 0x20

struct filter_info {
   int syscall_nr;                  // Syscall number
   char comm[TASK_FILTER_LEN]; // Process name
};
```

Syscall filtering mechanism

To block the execution of a specified system call, you need to intercept the system call and modify its behavior. This can be done by hooking the system call. Your implementation should:

- **Intercept the target system call**:
  Modify the syscall table entry to point to a custom handler instead of the original syscall handler.
- **Check process identity**:
  When the syscall is invoked, verify whether the calling process matches the specified process name.
- **Enforce syscall filtering**:
  If a match is found, return an appropriate error code to prevent execution. Otherwise, forward the call to the original syscall handler.

Your implementation must include a data structure that stores the syscall filter information, which should at least contain:

- The system call number to be blocked.
- A reference to the original system call handler.
- Additional metadata if needed for filtering.

To meet requirements in this section, you must successfully hook either the `read` or `write` system call for a process. The correctness of your implementation will be evaluated based on whether:

- Successfully hooking the `read` or `write` system call for a process at a time.
- Ensure that the filtering mechanism correctly identifies and blocks syscalls for the specified process.
- Return an appropriate error code when blocking a syscall.
- Restore system calls when filters are removed or the kernel module removed to prevent unintended behavior in the kernel.

Example

The following example demonstrates how a user-space program interacts with the syscall filtering mechanism. It sets a filter to block the `write` syscall for a process named `xxx` and later removes the filter.

```c
int main() {
    int fd;
    struct filter_info info;
    ...
    info.syscall_nr = __NR_write;
    strncpy(info.comm, "xxx", TASK_FILTER_LEN - 1);
    info.comm[TASK_FILTER_LEN - 1] = '\0';

    ioctl(fd, IOCTL_ADD_FILTER, &info);
    printf("hello world!\n"); // This should be blocked
    ioctl(fd, IOCTL_REMOVE_FILTER, &info);
    printf("hello world!\n"); // This should successfully print "hello world!"
}
```

Hint 1: In the AArch64 kernel, syscall table (sys_call_table) is defined in **arch/arm64/kernel/sys.c** as a constant. It is located in the **.rodata** segment in the compiled kernel binary, which is a read-only segment. To circumvent this, you may use **update_mapping_prot** to update the Write permission of the memory segment.

Hint 2: Since your rootkit is compiled as a kernel module, it does not have access to some kernel symbols. Linux only exports a certain set of symbols to module developers. For missing symbols, you could use **kprobe** to search for the function **kallsyms_lookup_name**, which will be useful to find other missing kernel symbols.

The following two references introduce how kprobe works and how to use:

- An introduction to KProbes
- Kernel Probes (Kprobes)

Hint 3: The symbols (functions or global variables) defined in your module will go away after you remove the loaded module. The kernel will not have access to the symbols.

Hint 4: As we discussed in the lecture, system call parameters are passed via general purpose registers in Arm (e.g. x0, x1, x2). You might want to figure out how system call parameters were passed by the kernel originally.

Questions (10%)

Answer the following questions in your write-up:

- The system call filtering based approach is vulnerable to the so called Returned Oriented Programming (ROP) attack, why is that? Name a concrete attack example in your write-up, and discuss approaches that you could use to address the vulnerabilities.
- What are the advantages and disadvantages of the filtering approach served by the module in this assignment compared to an approach based on `ptrace`?

Bonus (10%)

Extend your syscall filtering implementation to support more flexible filtering. Your solution should:

- Hook syscalls for **any** process, not just the current one.
- Filter additional syscalls beyond `read` and `write`. (e.g., `getdents64`).
- Support multiple syscall filters per process and the same syscall filter across multiple processes simultaneously.

Please explain the implementation in your write-up. Additionally, include a **test program** that demonstrates:

- Blocking multiple syscalls for a single process.
- Blocking specific syscalls across multiple processes.

Write-up (10%)

You are required to provide a write-up for the assignment. The write-up should include:

- An explanation of your source code (e.g., its functionality).
- A detailed description of how you test the rootkit.
- Answers to the questions listed in the `Filter syscall` section.

Homework submission

You should submit the assignment via NTU Cool.

Submission Format

For the rootkit, you will be required to submit source code, a Makefile, and a write-up file. Compress all the files in `hw1.zip`, and upload the zip file to NTU Cool.

Your `Makefile` does not have to deal with copying or installing the rootkit in your VM. We recommend you use the default Makefile that we provide.

```
.
├── source code of your userspace test program
├── Makefile
├── write-up.md
├── rootkit.c
├── rootkit.h
├── any extra file
```

Grading criteria

You will get zero points if your code fails to compile. Please run checkpatch against the source code for your rootkit module.

You are required to properly manage resources (free allocated memory), handle errors, and return error codes to user space.

Plagiarism policy

You are allowed to reference sources from the internet. If you do, please attach all of your references in the write-up. If we find that your code is similar to other's from the internet, we will count it as plagiarism if we cannot find the corresponding references. You will automatically get zero points for the assignment.

Generating/producing code/text for the assignment with generative AI tools, such as ChatGPT, is considered plagarism. If we catch you doing that, you will get zero points.

Late Policy

We do not accept late submissions for this assignment. Please start the assignment ASAP.